Requêtes HTTP

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Qu'est-ce-qu'une requête HTTP ? **3**

A. Définitions préliminaires....3

B. Pourquoi en a-t-on besoin ?....4

C. Fonctionnement d'une requête HTTP....4

D. Sécuriser une connexion HTTP : le protocole HTTPS....6

E. Exercice : Quiz....7

III. Réaliser des requêtes HTTP en Python **8**

A. De quoi a-t-on besoin ?....8

B. Quelques outils utiles....8

C. Exercice : Quiz....8

IV. L'essentiel **9**

V. Auto-évaluation **10**

A. Exercice....10

B. Test....10

Solutions des exercices **11**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python

Contexte

Il est souvent nécessaire pour une application de devoir communiquer avec le monde extérieur. Qu'il s'agisse d'une application Web, Windows, Linux, etc.

Nous aborderons dans ce cours les requêtes HTTP. Ce type d'échange est, à l'heure actuelle, le moyen le plus répandu (et un des plus simples) pour communiquer ou échanger des données avec une entité logicielle, qu'elle soit située sur le Web ou non.

Nous verrons comment mettre en place ce type d'échange, à quoi il faut faire attention, et nous donnerons quelques outils utiles pour pouvoir les exploiter efficacement. Vous verrez qu'une nouvelle fois, Python nous simplifiera la vie !

II. Qu'est-ce-qu'une requête HTTP ?

A. Définitions préliminaires

Le protocole HTTP (Hypertext Transfer Protocol) est une notion de base largement répandue sur le Web. Il s'agit tout simplement d'un protocole de communication client-serveur. Lorsque vous visitez une page Web, le chargement de cette page dans votre navigateur utilise, par exemple, le protocole HTTP.

Aujourd'hui, on a plus souvent affaire au protocole HTTPS, le 'S' étant l'acronyme de 'Secure'. Il s'agit dans ce cas d'un protocole HTTP chiffré. Vous utilisez le protocole HTTPS lorsque vous accédez par exemple à votre banque en ligne, ou tout autre site web de e-commerce.

Le protocole HTTP fait partie de la couche applicative du modèle OSI définissant l'empilement complet d'une architecture réseau, c'est la couche la plus élevée, qui manipule des données « *lisibles* » : une page web, du texte, un fichier, etc. Il existe d'autres protocoles applicatifs bien connus, comme par exemple le FTP (« *File Transfer Protocol* »).

Le modèle OSI est l'acronyme de 'Open systems interconnection'. Il définit de manière conceptuelle les différentes couches qui interviennent dans un système de communication. Son parallèle, le modèle TCP/IP, est en quelque sorte une version simplifiée. Sur la figure ci-dessous, le modèle OSI est à gauche, tandis que le modèle TCP/IP est à droite.

Application	HTTP, DNS, DHCP, FTP	Application
Présentation		
Session		
Transport	TCP, UDP	Transport
Réseau	IPv4, IPv6, ICMPv4, ICMPv6	Internet
Liaison de données	Ethernet	Accès réseau
Physique		

Source : The Tech Adventurer - Comprendre modèles OSI et TCP/ IP[1]

B. Pourquoi en a-t-on besoin ?

Le protocole HTTP est devenu, avec le temps et avec l'avènement du web, une façon quasiment incontournable d'échanger des données.

Ainsi, même si le protocole HTTP trouve son origine dans le chargement des pages web, son utilisation est même très répandue dans les échanges client-serveur qui n'impliquent pas forcément l'utilisation de pages web.

Pour un serveur, exposer ses données via le protocole HTTP (c'est ce qu'on rencontre très souvent sous la dénomination 'API REST', qu'on aura l'occasion de voir dans un prochain cours), est une façon simple et efficace pour les rendre disponibles à une application quelconque (supervision, tableau de bord, application Python, etc.

Par ailleurs, le protocole HTTP ne fonctionne pas que sur le web. On peut ainsi avoir des échanges selon le protocole HTTP entre deux entités logicielles qui ne sont pas interconnectées par le Web (par exemple, elles peuvent tourner sur une même machine, ou être reliées à travers un réseau local).

C. Fonctionnement d'une requête HTTP

Le protocole HTTP est construit sur base d'un système requête-réponse. Le client envoie ainsi une requête au serveur, qui lui répond.

Les requêtes peuvent être de différents types :

- GET : récupérer des données en lecture uniquement
- POST : modifier les données du serveur, ou en créer des nouvelles
- PUT : mettre à jour des données présentes sur le serveur
- DELETE : supprimer des données du serveur

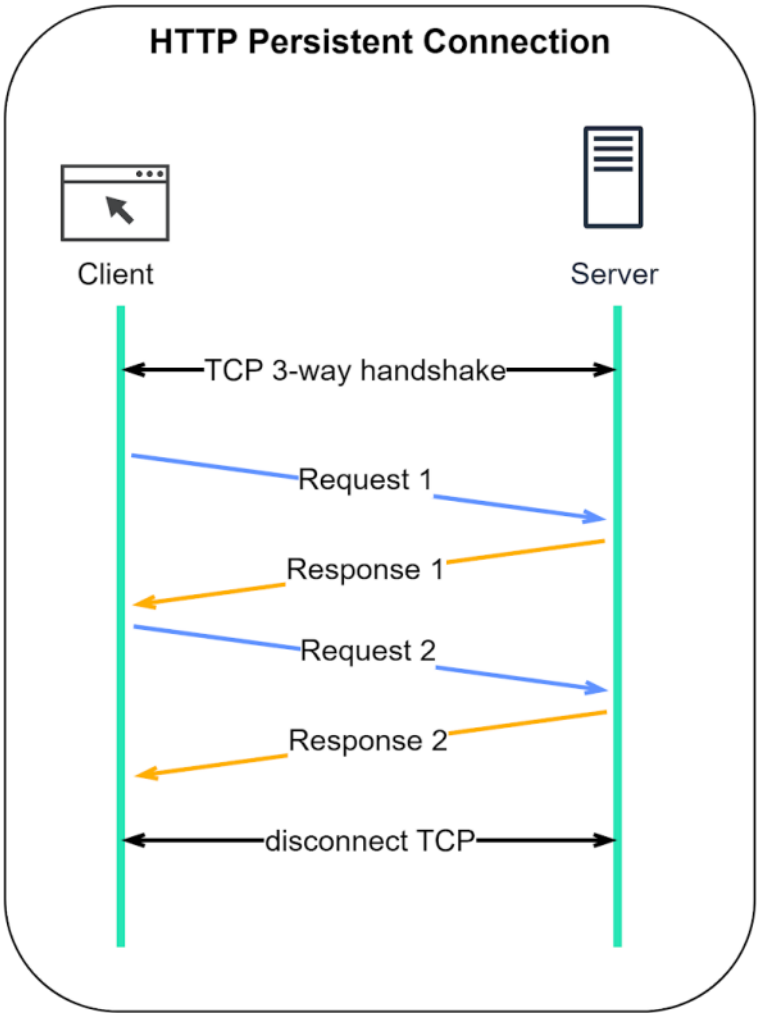
1 https://thetechadventurer.com/articles/comprendre-modeles-osi-tcp-ip/

Tous droits réservés à STUDI - Reproduction interdite

- PATCH : modifier partiellement des données du serveur
- TRACE : renvoie vers le client sa requête initiale (débogage)
- OPTIONS : récupérer les options du serveur

Cependant, deux requêtes sont principalement utilisées : GET et POST, pour lire ou écrire.

Les requêtes sont envoyées à partir du moment où la connexion entre le client et le serveur est établie, c'est ce qu'on appelle le « TCP handshake ». Pour les plus curieux, le protocole TCP est le type de transport utilisé par le protocole HTTP (bien que théoriquement, le protocole HTTP puisse être utilisé sur un transport UDP).



Source : The Foundation of REST API: HTTP[1]

Sur l'image ci-dessus, on voit que l'échange HTTP se passe de la manière suivante :

1. La connexion (TCP) est établie entre le client et le serveur.
2. Une requête est envoyée par le client au serveur.
3. Celui-ci répond avec :
    - Les données demandées par le client lors d'une lecture (GET).
    - Une confirmation que les données ont été correctement prises en compte (POST).
4. Plusieurs requêtes successives peuvent être envoyées sur la même connexion.

1 https://blog.bytebytego.com/p/the-foundation-of-rest-api-http

Tous droits réservés à STUDI - Reproduction interdite

5. Si aucune requête n'est faite dans un temps imparti, la connexion est arrêtée. Si un nouvel échange doit avoir lieu, la connexion sera rétablie.

D. Sécuriser une connexion HTTP : le protocole HTTPS

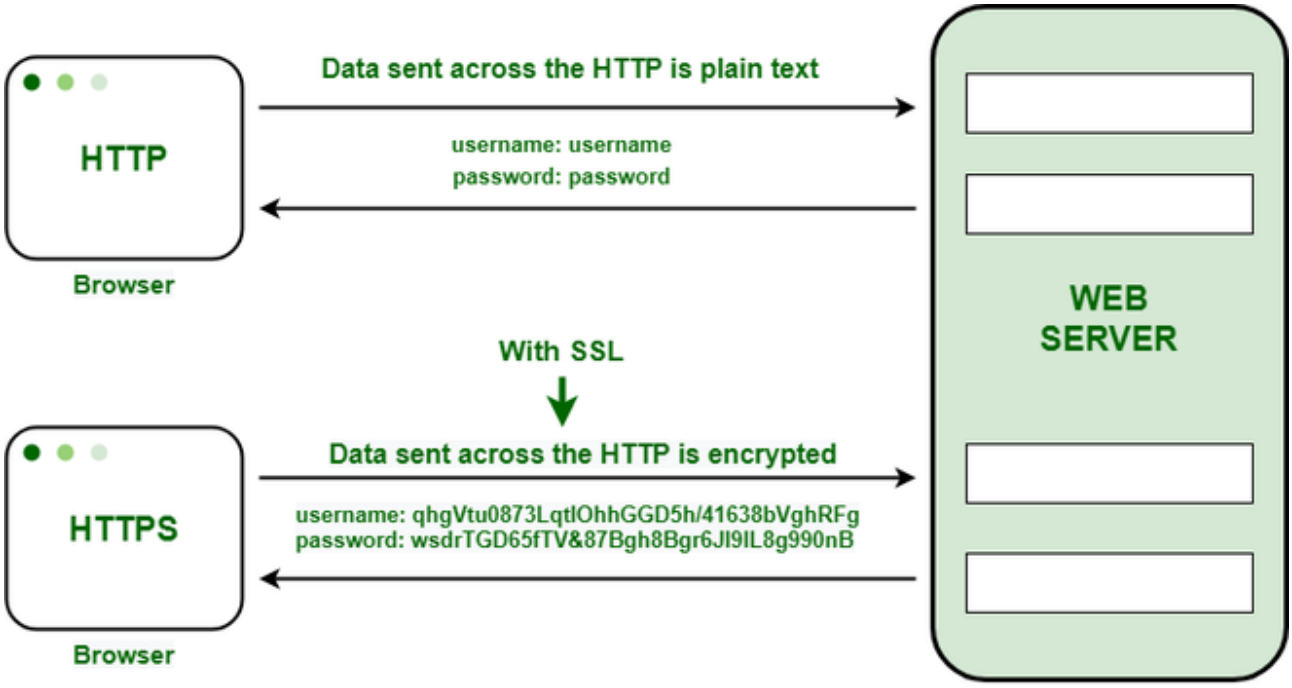
Comme nous l'avons vu, le protocole HTTP standard ne fournit aucune sécurité : les données échangées transitent en clair entre le client et le serveur. Dans un réseau local, cela ne pose généralement pas de problème (bien que la tendance soit récemment entrain de changer), mais lorsque les données sont échangées à travers un réseau connecté au web, cela peut potentiellement poser de graves problèmes en matière de sécurité.

Imaginez par exemple que vous devez vous authentifier sur un site web qui ne fournit pas une connexion HTTPS, mais une simple connexion HTTP.

Vous arrivez sur la page d'authentification, vous entrez votre login et mot de passe, et appuyez sur le bouton de connexion. À ce moment là, un échange HTTP contenant votre login et mot de passe, permet au site web de vérifier si vous existez dans sa base de données, pour pouvoir vous authentifier.

Si une personne malveillante intercepte cet échange, il aura ainsi accès à vos données sans aucune difficulté. Imaginez ce que cela peut avoir comme impact dans les échanges bancaires par internet.

Pour remédier à cela, il existe le protocole HTTPS. Le protocole HTTPS est identique au protocole HTTP, sauf qu'une couche de chiffrement et de vérification d'authenticité ont été ajoutées. Ainsi, le serveur doit présenter un certificat valide, et les données qui transitent entre le client et le serveur sont en permanence cryptées :



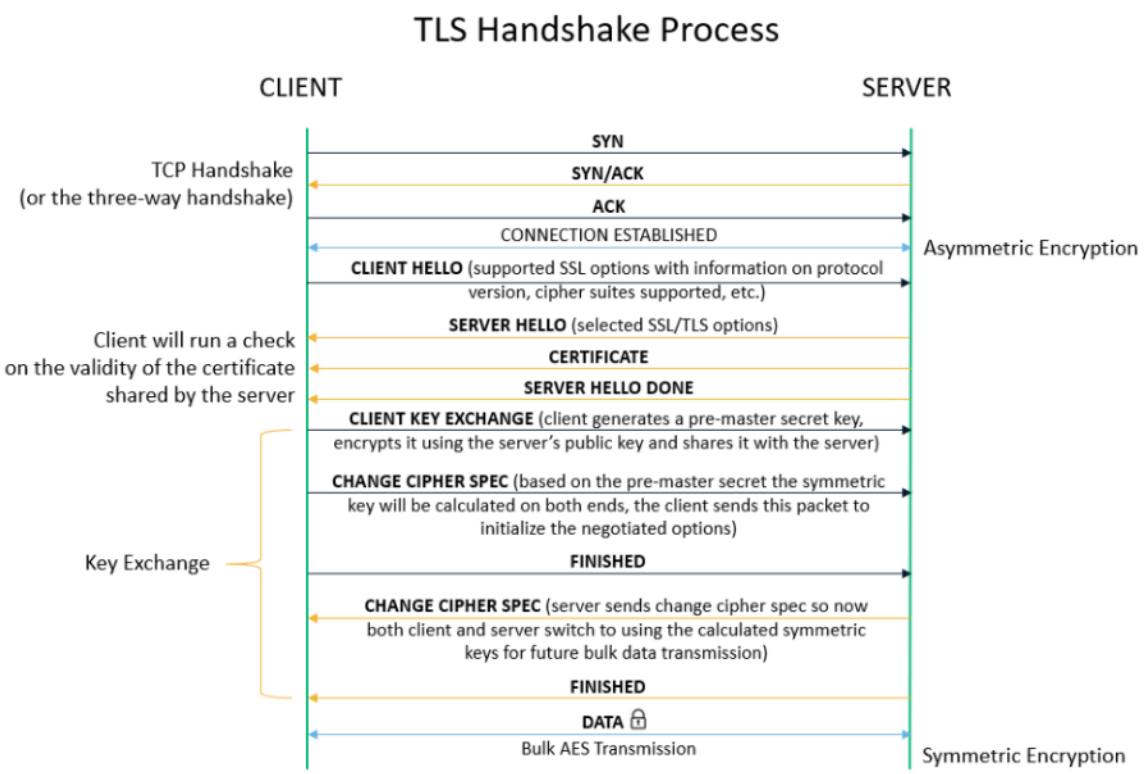
Source : Difference between http:// and https://[1]

Les protocoles de chiffrement (c'est-à-dire l'algorithme utilisé pour crypter les données transmises), sont généralement le SSL et le TLS. Nous n'allons pas entrer dans les détails, mais rappelez-vous seulement que le SSL est plus ancien et moins sécurisé que le TLS, qui est maintenant devenu la norme.

1 https://www.geeksforgeeks.org/difference-between-http-and-https/

Tous droits réservés à STUDI - Reproduction interdite

Le protocole HTTPS est autrement plus complexe que le HTTP. À titre de comparaison avec ce que nous venons de voir pour le protocole HTTP, voici les actions mises en œuvre dans un échange HTTPS :



Source : How Does HTTPS Work to Improve Website Security?[1]

Nous n'allons pas entrer dans tous ces mécanismes, qui sont hors contexte par rapport à ce cours. Cependant, cela vous permet de vous rendre compte des couches supplémentaires de sécurité ajoutées dans le cadre du protocole HTTPS par rapport au protocole HTTP.

E. Exercice : Quiz

[solution n°1 p.13]

Question 1

Le protocole HTTP définit…

- ○ Le protocole de sécurité d'une connexion à un serveur
- ○ Le protocole de transport utilisé dans le cadre d'une connexion distante
- ○ Le protocole de communication utilisé dans le cadre d'une connexion distante

Question 2

Avec le protocole HTTP, le transport UDP ne peut jamais être utilisé.

- ○ Vrai
- ○ Faux

Question 3

1 https://sectigostore.com/page/how-does-https-work/

Tous droits réservés à STUDI - Reproduction interdite

Le HTTP est dangereux pour la sécurité.

- Vrai
- Faux

Question 4

Le protocole HTTP n'est utilisables que si une connexion internet est disponible.

- Vrai
- Faux

Question 5

Lorsque je consulte un site sensible (banque, etc.), une connexion HTTPS est systématiquement mise en place.

- Vrai
- Faux

III. Réaliser des requêtes HTTP en Python

A. De quoi a-t-on besoin ?

En Python, pour réaliser des requêtes HTTP, il suffit d'installer le module 'requests' : c'est aussi simple que cela. Dans votre environnement virtuel, tapez simplement la commande suivante :

```
conda install requests
```

Ou bien, si vous utilisez *pip* :

```
pip install requests
```

Une fois *requests* installé, il suffira comme toujours de l'importer dans votre application Python :

```
import requests
…
…
```

Nous voilà enfin prêts à réaliser des requêtes HTTP(s) !

B. Quelques outils utiles

Comme vous avez pu le voir, des requêtes HTTP nécessitent un client (notre application), mais également un serveur.

Il existe plusieurs serveurs sur le web, capables de nous aider a tester nos requêtes, et tout ça gratuitement !

Nous allons voir deux outils en ligne qui vont nous aider dans la mise en œuvre de nos requêtes HTTP : Beeceptor et Reqbin.

Beeceptor va servir de serveur. Il 'intercepte' (d'où son nom) nos requêtes et nous permet de vérifier si le contenu de celles-ci sont correctes.

De son coté, *Reqbin* est un outil « *client* », qui va nous permettre de tester nos requêtes avant de les coder en Python.

C. Exercice : Quiz

[solution n°2 p.14]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

HTTP est l'acronyme de :

- HyperTransfer Protocol
- Hypertext Transfer Protocol
- Hype Transfer Protocol

Question 2

La façon la plus efficace pour tester des requêtes en Python est de les programmer directement dans l'éditeur (PyCharm, VSCode, etc.)

- Vrai
- Faux

Question 3

Il est possible d'envoyer des fichiers à travers le protocole http.

- Vrai
- Faux

Question 4

http est le seul protocole de la couche applicative dans le modèle OSI.

- Vrai
- Faux

Question 5

Le protocole http est très largement utilisé, dans tous les types d'applications, online ou offline.

- Vrai
- Faux

IV. L'essentiel

Le protocole de communication HTTP est absolument incontournable dans les langages de programmation. Il permet de manière générale, d'échanger des données entre deux entités logicielles, qu'elles soient exécutées :

- Sur une seule et même machine
- Sur deux machines dans un réseau local
- Sur deux machines dans un réseau distant, reliées par internet

Nous avons vu des notions importantes, liées à la sécurité ainsi qu'à l'authentification.

La librairie 'requests' permet de réaliser des requêtes de manière extrêmement simple, avec toute une série de paramètres qui permettent de couvrir tous les cas de figure qu'on puisse imaginer.

Les requêtes HTTP sont un moyen simple, pour une application, de mettre à disposition des informations. Nous verrons dans un prochain cours que la mise à disposition de ces informations passe généralement par une standardisation des interfaces, qu'on appelle « *API REST* ».

Tous droits réservés à STUDI - Reproduction interdite

V. Auto-évaluation

A. Exercice

Vous disposez d'un serveur WEB, qui met à disposition les informations suivantes :

- Le nombre d'utilisateurs actuellement en ligne : http://www.my-website.com/users-online
- La charge du serveur en pourcentage : http://www.my-website.com/server-load

Les données vous sont renvoyées au format json :

```
1 {
2         'data': x
3 }
```

Où **x** est la donnée (nombre d'utilisateurs ou pourcentage de charge).

Question 1

[solution n°3 p.14]

Vous devez mettre en place une application de supervision. Cette application doit périodiquement (environ une fois par minute) récupérer les informations mises à disposition par le serveur.

Question 2

[solution n°4 p.15]

Vous vous rendez compte que n'importe qui possédant ces URLS peut faire des requêtes et ainsi obtenir ces informations. Quelles sont les mesures que vous devriez mettre en œuvre pour sécuriser vos requêtes ?

B. Test

Exercice 1 : Quiz

[solution n°5 p.14]

Question 1

HTTP est l'acronyme de :

- ○ HyperTransfer Protocol
- ○ Hypertext Transfer Protocol
- ○ Hype Transfer Protocol

Question 2

La façon la plus efficace pour tester des requêtes en Python est de les programmer directement dans l'éditeur (PyCharm, VSCode, etc.)

- ○ Vrai
- ○ Faux

Question 3

Il est possible d'envoyer des fichiers à travers le protocole http.

- ○ Vrai
- ○ Faux

Question 4

http est le seul protocole de la couche applicative dans le modèle OSI.

- ○ Vrai
- ○ Faux

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Le protocole http est très largement utilisé, dans tous les types d'applications, online ou offline.

- ○ Vrai
- ○ Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 7 Solution n°1

Question 1

Le protocole HTTP définit…

- ○ Le protocole de sécurité d'une connexion à un serveur
- ○ Le protocole de transport utilisé dans le cadre d'une connexion distante
- ◉ Le protocole de communication utilisé dans le cadre d'une connexion distante

Le protocole HTTP est un protocole de communication. Dans le modèle OSI, il est situé au dessus de la couche de transport.

Question 2

Avec le protocole HTTP, le transport UDP ne peut jamais être utilisé.

- ○ Vrai
- ◉ Faux

Bien que rare, c'est théoriquement possible. La couche de transport et la couche applicative (qui spécifie notamment le protocole HTTP), ne sont pas contraintes l'une par l'autre.

Question 3

Le HTTP est dangereux pour la sécurité.

- ◉ Vrai
- ○ Faux

En effet, c'est généralement vrai (sauf pour de rares cas où l'utilisation est exclusivement locale). Le protocole HTTP tend d'ailleurs généralement à disparaître pour tous les sites Web usuels, au profit du HTTPS.

Question 4

Le protocole HTTP n'est utilisables que si une connexion internet est disponible.

- ○ Vrai
- ◉ Faux

C'est faux. Un échange HTTP peut se produire entre deux applicatifs, l'un agissant en tant que client, et l'autre en temps que serveur. Quand c'est le cas, l'adresse cible utilisée est l'adresse locale :

127.0.0.1. S'il s'agit d'un réseau local (LAN), l'adresse de la machine cible sur le réseau local sera utilisée.

Question 5

Lorsque je consulte un site sensible (banque, etc.), une connexion HTTPS est systématiquement mise en place.

- ◉ Vrai
- ○ Faux

Oui, c'est évidemment le cas. À l'heure actuelle, la plupart des navigateurs Web bloquent les accès aux sites web qui demandent des informations d'authentification, si le site web ne présente pas les sécurités nécessaires (certificat, chiffrement).

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 8, 10 Solution n°2

Question 1

HTTP est l'acronyme de :

- ○ HyperTransfer Protocol
- ◉ Hypertext Transfer Protocol
- ○ Hype Transfer Protocol

Q HTTP signifie 'Hypertext Transfer Protocol'. Il s'agit d'un nom étroitement lié aux sites web, et leurs liens hypertextes (URLs)

Question 2

La façon la plus efficace pour tester des requêtes en Python est de les programmer directement dans l'éditeur (PyCharm, VSCode, etc.)

- ○ Vrai
- ◉ Faux

Q Des outils en ligne et gratuits existent pour mettre au point et tester ses requêtes. Nous avons vu beeceptor et Reqbin, qui permettent de tester l'envoi et la réception des requêtes http.

Question 3

Il est possible d'envoyer des fichiers à travers le protocole http.

- ◉ Vrai
- ○ Faux

Q Oui, à l'aide d'une requête de type post, en renseignant le champ 'data', il est possible d'envoyer directement un fichier.

Question 4

http est le seul protocole de la couche applicative dans le modèle OSI.

- ○ Vrai
- ◉ Faux

Q Il existe d'autres protocoles applicatifs, comme par exemple le bien connu « *FTP* » (« *File Transfer Protocol* »).

Question 5

Le protocole http est très largement utilisé, dans tous les types d'applications, online ou offline.

- ◉ Vrai
- ○ Faux

Q C'est vrai. Le protocole HTTP est extrêmement souvent employé pour échanger des données.

p. 10 Solution n°3

Tous droits réservés à STUDI - Reproduction interdite

Le code suivant répond au besoin :

```
1 import requests
2 import time
3
4 nUsersAddress = "http://www.my-website.com/users-online"
5 serverLoadAddress = "http://www.my-website.com/server-load"
6
7 while(True):
8     try:
9         nUsers = requests.get(nUsersAddress, timeout = 5)
10        serverLoad = requests.get(serverLoadAddress, timeout = 5)
11        print(nUsers.json()["data"]["data"])
12        print(serverLoad.json()["data"]["data"])
13        time.sleep(60)
14    except Exception as e:
15        print(f"Erreur : {e}")
```

Le résultat de la requête est d'abord convertit en objet json. Ensuite, dans le contenu (« *data* »), nous récupérons notre objet Json. On doit encore prendre l'élément « *data* », pour avoir accès à la donnée finale.

L'appel à time.sleep() bloque notre programme pendant 60 secondes avant de recommencer. Notez que dans ce cas, si chaque requête prend plusieurs secondes, le délai d'exécution entre deux itérations sera supérieur à 60 secondes.

p. 10 Solution n°4

La première chose à faire est de rendre votre site web compatible avec le protocole HTTPS. Pour ce faire, vous devez généralement obtenir un certificat d'authenticité, qui permettra à votre client de valider la connexion.

Ensuite, vous devriez mettre en place une authentification. Le plus simple est de générer des tokens d'authentification, qui seront par exemple valables pendant quelques jours, et renouvelés systématiquement. Vous pourrez ainsi utiliser ce token dans votre application sans devoir manipuler de nom d'utilisateur ou de mot de passe.

Tous droits réservés à STUDI - Reproduction interdite